

"The Complete Guide to pstack Pt. 1" / X

 x.com/poteto/status/2094457600259842065

2026年8月20日

The Complete Guide to pstack Pt. 1
pstack 完整指南 第一部分

[846K](#)

In this series of posts, I'm going to show you how I use

[pstack](#)

, my personal set of skills for doing rigorous engineering work. It's allowed me to ship 2,000 PRs a month to production with high confidence.

Personally, I have never put much emphasis into how many lines of code or how many PRs I was landing. Before agents, no one cared, and rightfully so, as raw productivity did not always equate to quality or a visible outcome for users. It was simply a vanity metric. 就我个人而言，我从来不太在意自己写了多少行代码或者提交了多少个 PR。在代理出现之前，没人关心这些，这也不无道理，因为单纯的生产力并不总是等同于质量或用户可见的成果。这只不过是一个虚荣的指标而已。

But I've discovered through the course of building pstack that volume does matter, especially when you are able to maintain or even increase the level of quality of the product with agents. For example, I started working on Grok

[@Bot](#)

about 2 months ago, when it was still in its early days and the codebase was fresh but starting to grow. Despite the team growing and now landing hundreds of PRs a day into the Grok

[@Bot](#)

codebase, pstack has allowed me to keep the quality of the code high for everyone as I constantly monitor code, refactor, add new lints and checks, and also work on features. 但在构建 pstack 的过程中，我发现数量确实很重要，尤其是在能够通过代理保持甚至提高产品质量的情况下。例如，我开始开发 Grok。

[@Bot](#)

大约两个月前，Grok 项目还处于早期阶段，代码库虽然很新，但已经开始发展壮大。尽管团队不断壮大，现在每天都有数百个 PR 提交到 Grok 项目，但 Grok 项目仍然处于起步阶段。

[@Bot](#)

codebase 和 pstack 让我能够保持代码的高质量，因为我可以不断监控代码、重构代码、添加新的代码检查和测试，还可以开发新功能。

every team needs a gardener. someone quietly watching the stream of PRs flowing into your codebase, noticing the smells: the third isRecord this week, the lint suppressions creeping like ivy across your carefully planned garden. a steady hand tending the weeds

that would engulf

每个团队都需要一个园丁。一个默默观察着流入代码库的 PR 流，留意着其中的异味的人：比如这周第三个 isRecord，还有像常春藤一样蔓延在你精心规划的花园里的 lint 抑制。一个稳健的手，照料着那些即将吞噬一切的杂草。

organic architecture good codebases have always had strong foundations and constraints. these files go here. this kind of code goes there. the codebase is held together either by catering to the lowest common denominator developer using conventional frameworks and languages, or

有机建筑 优秀的代码库总是拥有坚实的基础和明确的约束。这些文件放在这里，这种类型的代码放在那里。代码库的维系要么是迎合使用传统框架和语言的最低标准开发者，要么是.....

Being Grok

[@Bot](#)

's gardener and maintainer is something I was only able to do through pstack. Our early momentum after building the prototype was very high and many people were joining the team. I had a critical moment of opportunity to refactor the whole codebase, while it was being built and extended and with no downtime, into something with strong foundations. A codebase with high quality that scales no matter how many engineers (and most importantly, non-engineers) contribute to it. All of this work requires me to refactor and improve the foundations of Grok Bot as it's being built, and you can only do that when the foundations can keep up with the number of contributions.

成为格洛克

[@Bot](#)

我之所以能够担任 Grok Bot 的园丁和维护者，完全得益于 pstack 平台。原型构建完成后，我们早期发展势头强劲，许多人加入了团队。我抓住了一个关键的机遇，在代码库不断构建和扩展的同时，无需停机，就能将其重构为一个拥有坚实基础的平台。这个高质量的代码库能够适应各种规模的扩展，无论有多少工程师（更重要的是，非工程师）参与贡献。所有这些工作都需要我在 Grok Bot 构建的过程中不断重构和改进其基础架构，而只有当基础架构能够跟上贡献数量的增长时，这一切才能实现。

Grok Bot is one of the most efficient and performant AI desktop apps out on the market

Grok Bot 是市场上最高效、性能最强的 AI 桌面应用程序之一。

The proof is in Grok

[@Bot](#)

itself. Over the next few weeks, I'll tell you everything you need to know to be able to build and maintain a high quality app using pstack.

证据就在 Grok 中

[@Bot](#)

就它本身而言。在接下来的几周里，我将告诉你使用 pstack 构建和维护高质量应用程序所需的一切知识。

Part 1 – Verification is all you need

第一部分——验证就足够了

The most critical skill to have in your toolbox is a high quality verification skill. This skill is so important to have and maintain that I think of it more like critical infrastructure rather than "just" a skill. A good one will amplify the output of your whole team, including non-engineers. Done well, you will 100-1000x your whole team's output.

工具箱中最关键的技能莫过于高质量的验证技能。这项技能至关重要，需要不断学习和维护，因此我认为它更像是关键基础设施，而不仅仅是一项技能。优秀的验证技能能够显著提升整个团队的产出，包括非工程师成员。如果运用得当，团队的产出甚至可以提升 100 到 1000 倍。

If you're not familiar with the term, verification means that an agent can verify its own work. It can keep going until it succeeds at its task, because it can now close the loop without you being the bottleneck. If you're interested to know more of the story of how I created my first verification skill for Cursor, check out my previous post

[Loops You Can Trust](#)

如果你不熟悉这个术语，验证指的是代理可以验证自己的工作。它可以持续运行直到任务成功，因为它现在可以闭环运行，而不会受到你的阻碍。如果你想了解更多关于我如何为 Cursor 创建第一个验证技能的故事，请查看我之前的帖子。

[值得信赖的循环](#)

Let's build a verification skill together

让我们共同培养验证技能。

To start, install pstack and then run

[/create-verification-skill](#)

. I also recommend adding

[Dr Eggbot](#)

, my bot that helps you create high quality bots, to your roster. Dr Eggbot ships with

[pstack](#)

. It'll teach coding bots how to use it, and it can also make non-coding bots with the same rigor.

首先，安装 pstack，然后运行

[创建验证技能](#)

我还建议添加

[蛋机器人博士](#)

我的机器人 Dr Eggbot 可以帮助您创建高质量的机器人，并将其添加到您的机器人阵容中。

Dr Eggbot 附带

[pstack](#)

它将教会编程机器人如何使用它，也可以制造出同样严谨的非编程机器人。

You can ask Dr Eggbot to create an engineer bot for you that you can then ask to run `/create-verification-skill` and set up a daily routine to run `/maintain-verification-skill`.

您可以要求 Eggbot 博士为您创建一个工程师机器人，然后您可以要求该机器人运行 `/create-verification-skill` 命令，并设置一个每日例程来运行 `/maintain-verification-skill` 命令。

love Dr Eggbot

爱蛋机器人博士

While that runs, let's walk through what the skill does and how it makes a high quality verification skill for you.

在它运行的同时，让我们一起来看看这项技能的作用，以及它如何为你打造一项高质量的验证技能。

I distilled all of our verification skills that we use to build Grok

[@Bot](#)

and Cursor into this skill as a sort of meta-skill. It teaches your agent how to create a high quality one for your own app.

Now this is where choice of tech stack is important. If you're building an app in Electron or for the web for example, you can take advantage of the rich debugging tools available for the JS ecosystem. For example, the

[Chrome DevTools Protocol \(CDP\)](#)

[Chrome 开发者工具协议 \(CDP\)](#)

allows you to use the same tooling available in your browser's developer tools. Or if you're building an iOS app, making use of the simulator.

You ideally want the ability to interact with your app, debug it, take perf traces, and any other debugging and development tooling that you might typically use if you were developing the app by hand. If you don't have a rich runtime to make use of, you may need to ask your agent to create tools for you (eg using lldb, or a custom package that runs as a sidecar in dev environments), or just make use of what you have available.

理想情况下，您需要能够与应用程序交互、调试应用程序、获取性能跟踪信息，以及使用任何其他您通常在手动开发应用程序时会用到的调试和开发工具。如果您没有功能丰富的运行时环境可用，您可能需要请您的代理为您创建工具（例如使用 lldb 或在开发环境中作为 sidecar 运行的自定义包），或者直接使用您现有的资源。

I personally feel that agentic verification is so important that I would unironically suggest building your own rich debugging tools, or even choosing a different tech stack, in order to have unfair advantages and extreme productivity in building software. As I mentioned earlier, giving agents the ability to verify their own work unlocks everyone in your organization to be able to contribute and validate that their changes actually work. The harder your tech stack is to debug and control, the more difficult it will be to use agents

productively.

我个人认为，代理验证至关重要，因此我会毫不犹豫地建议构建您自己的功能强大的调试工具，甚至选择不同的技术栈，以便在软件开发中获得绝对优势和极高的效率。正如我之前提到的，赋予代理验证自身工作的能力，能够让组织中的每个人都参与进来，并验证他们的更改是否真正有效。技术栈越难调试和控制，就越难高效地使用代理。

Make it Reproducible

使其可复现

In pstack, we have a principle called

["Build the Lever"](#)

[“建立杠杆”](#)

. What this means in the context of creating a skill, is that we prefer to give agents tools rather than just markdown. For verification skills, this means creating a small CLI that scripts interaction and debugging of your app in a small, agent friendly utility. This means that agents consume fewer tokens trying to do a task (run a CLI command instead of writing a throwaway script to click on something), and makes your verification skill more reproducible and testable.

Here's a hypothetical example of a CLI your agent might make for an Electron app:

以下是一个假设的 CLI 示例，您的代理可能会为 Electron 应用程序创建该 CLI：

```
# health
node .cursor/skills/verify-atlas/control-atlas.mjs doctor

# open a blank thread and send
node .cursor/skills/verify-atlas/control-atlas.mjs new-session
node .cursor/skills/verify-atlas/control-atlas.mjs send "list open tasks in this project"

# keyboard path
node .cursor/skills/verify-atlas/control-atlas.mjs press "Meta+KeyN"

# accessibility snapshot of the live UI
node .cursor/skills/verify-atlas/control-atlas.mjs snapshot

# screenshot for evidence
node .cursor/skills/verify-atlas/control-atlas.mjs screenshot /tmp/atlas-proof.png

# wait for streaming / layout to settle
node .cursor/skills/verify-atlas/control-atlas.mjs wait-settle

# flip a feature flag for the session
node .cursor/skills/verify-atlas/control-atlas.mjs feature-flag rooms_v2 on
```

Now, all agents can use this CLI to quickly navigate and debug your app. You'll also want to start thinking about the dev experience of building your app:

现在，所有客服人员都可以使用此命令行界面快速浏览和调试您的应用。您还需要考虑应用开发过程中的用户体验：

- seeding a dev database
- how to handle auth, test users, API calls against a test/staging environment
- installing and bringing up your dev environment in a consistent way

All of this is stuff you've probably needed to think about anyway when you were writing code yourself. So think of this as your agents' main utility for doing dev work on your app. Keep it well maintained and tested!

所有这些问题，你当初自己写代码的时候可能都考虑过。所以，你可以把这个功能看作是您的代理在应用开发中的主要工具。务必保持它的良好维护和测试！

Some other example commands you might want to consider:

您可能还想了解其他一些示例命令：

markdown

标记化

```
- **Inspection:** `info`, `snapshot`, `screenshot`, `components`  
- **Navigation:** `home`, `new-session`, `select-project`, `select-runtime`,  
`scroll`  
- **Interaction:** `send`, `click`, `click-xy`, `aria-click`, `type`, `press`,  
`eval`, `upload-image`, `add-context`, `feature-flag`  
- **Performance:** `trace`, `profile`, `record`, `perf-metrics`, `wait-settle`  
- **Streaming:** `console`, `network-log`, `network-summary`  
- **Health & cleanup:** `doctor`, `cleanup`, `watch --restart`
```

Once you have this basic setup, you should already start to see a big improvement in your agents. They should be able to navigate around and debug your app with ease.

完成这些基本设置后，您应该很快就能看到客服人员的表现有了显著提升。他们应该能够轻松地浏览和调试您的应用程序。

I recommend spending time here making this CLI good and error free before doing anything more advanced. You'll also want to think about (or ask your agent to) designing an agent friendly CLI. There are many resources online you can point your agent to, but the key properties I like are:

我建议您先花时间完善这个命令行界面 (CLI)，确保它运行良好且无错误，然后再进行更高级的操作。您还需要考虑（或者请您的代理人帮忙）设计一个便于代理人使用的 CLI。网上有很多资源可以推荐给您的代理人，但我比较看重的关键特性是：

- the API is easy to compose - think John Ousterhout's deep modules philosophy
API 易于组合——想想 John Ousterhout 的深度模块理念。

- any command with potentially destructive side effects should have a `--dry-run` option
任何可能产生破坏性副作用的命令都应该有 `--dry-run` 选项。
- make use of subcommands to gradually disclose functionality rather than all at once
使用子命令逐步揭示功能，而不是一次性全部揭示。
- error messages should be very descriptive and tell the agent what it should do instead
错误信息应该非常详细，并告诉代理程序应该如何操作。
- rich `--help` text
富文本 `--help`
- outputs returned in machine readable form (eg JSON)
以机器可读格式（例如 JSON）返回输出

When you've had some success running your verification skill to land a few PRs, you might start to wonder if you can parallelize more. For example, if an agent can now take your prompt and mostly drive it to a mergeable state, doesn't that free you up to run more agents?

当你的验证技能成功提交了一些 PR 后，你可能会开始思考是否可以并行处理更多任务。例如，如果一个代理现在可以接收你的提示并将其大部分引导至可合并状态，这岂不是可以让你腾出时间运行更多代理吗？

Your first instinct will be to add worktree support, meaning that your agents can use git to create a tracked copy of the repo where they can make changes in isolation to the main checkout. In theory, this lets you run multiple agents at once without their changes clobbering over each other.

你的第一反应可能是添加工作树支持，这意味着你的代理可以使用 Git 创建一个仓库的跟踪副本，并在该副本上独立地对主检出进行更改。理论上，这允许你同时运行多个代理，而不会导致它们的更改相互覆盖。

I would recommend against doing this. For one, it uses a lot of storage space and resources on your machine. You may be able to get away with running up to 10 agents in parallel with worktrees depending on the size of your repo and how powerful your machine is. But there's a far better way!

我不建议这样做。首先，它会占用大量存储空间和机器资源。根据代码库的大小和机器的性能，你或许可以并行运行最多 10 个代理程序和工作树。但还有更好的方法！

Cursor's

[cloud agents](#)

are agents that run on the cloud, on Cursor's infrastructure. These agents have access to a real computer, meaning that they can install dependencies, run your app, take videos and screenshots, and interact with your app like a real user can. If you've invested enough in the previous step to make your dev experience good, it shouldn't be a huge lift to be able

to set up cloud agents. When you first set up your cloud environment, we send an agent to help you get it set up and running correctly. After the first build, we take a [snapshot](#)

which means that subsequent cloud agent runs always start up quickly.

I highly recommend taking the time to set cloud agents up, as it unlocks a massive increase in productivity in parallelism. In a later post I'll show you how I run hundreds of subagents in parallel in the cloud! But for now, set up your environment and get it to a state where you can start to feel confident about running all your agents in the cloud.

我强烈建议您花时间配置云代理，因为它能大幅提升并行处理效率。在后续文章中，我会向您展示如何在云端并行运行数百个子代理！但现在，请先配置好您的环境，并使其达到您可以放心在云端运行所有代理的状态。

Keep agents smart with Feature Maps

利用特征地图提升代理商的智能水平

As your app grows more complex, agents need more guidance to be able to find features and interact with them. To do this, I've come up with something I call the Feature Map. As the name suggests, it's an easily searchable map of all the features available in your app, what it does, and how to get to it from a user's perspective.

随着应用变得越来越复杂，客服人员需要更多指导才能找到并使用这些功能。为此，我开发了一种叫做“功能地图”的工具。顾名思义，它是一张易于搜索的地图，展示了应用中所有可用功能、每个功能的作用以及如何从用户的角度访问它们。

Here's an

[example Feature Map](#)

[示例特征图](#)

that I've prepared for a fictional app called Atlas. It's just a couple of markdown file that are mentioned in the verification's SKILL.md.

You can put this file anywhere, but in `/create-verification-skill` we automatically create a `references/features` directory alongside a `README.md`. The readme is the map itself: a high level overview of all the major features available, with links to specific details. An example feature looks something like this:

您可以将此文件放在任何位置，但在 ``/create-verification-skill`` 目录下，我们会自动创建一个 ``references/features`` 目录，旁边还有一个 ``README.md`` 文件。``README`` 文件本身就是一张地图：它概述了所有主要功能，并提供了指向具体细节的链接。一个功能示例如下所示：

markdown

标记化

```
# Preferences
```

```
Full-screen preferences overlay and its tab set.
```

Sub-features

- settings-overlay: full-screen overlay opened from the gear or Cmd/Ctrl+,.
- settings-nav: left nav of tabs (General, Appearance, Models, Plan & Usage, ...).
- settings-search: in-overlay search (Cmd/Ctrl+K while settings is open).
- theme-picker: quick theme control on Appearance.

How to get to it (user POV)

Click the gear next to the account avatar, or press Cmd/Ctrl+,. Pick a tab from the left nav. Type in the preferences search box to jump. Escape or the close control dismisses.

Driving it with control-atlas

```
bash
node .cursor/skills/verify-atlas/control-atlas.mjs press "Meta+Comma"
node .cursor/skills/verify-atlas/control-atlas.mjs snapshot
node .cursor/skills/verify-atlas/control-atlas.mjs press "Escape"
```

- Overlay root: look for a dialog/region named Preferences in the ally tree.
- Tabs: click by visible name. Plan & Usage may be absent for some account states.
- While settings is open, Cmd/Ctrl+K is preferences search, not the global palette (see `multi-surface-journeys.md`).

Gotchas

- Closing settings mid-suite can leave focus nowhere useful. `new-session` or `home` recovers.
- Some tabs are entitlement-gated. Skip with an explicit account reason.

Don't worry about writing these yourself! When you run `/create-verification-skill`, your agent will automatically go through your app and catalog everything and create these references for you.

不用担心自己编写这些内容！运行 `/create-verification-skill` 后，您的代理会自动遍历您的应用程序，对所有内容进行分类并为您创建这些参考信息。

The Feature Map, when combined with the CLI, is one of the main reasons why pstack's verification skills are so good. Agents now have context about every single feature and how to get to it, saving precious tokens in its context window and teaching it exactly what it's for and how to get there.

特征图与命令行界面 (CLI) 的结合，是 pstack 验证能力如此强大的主要原因之一。代理现在能够获取每个特征的上下文信息以及如何访问该特征，从而节省上下文窗口中的宝贵令牌，并准确地了解每个特征的用途和访问路径。

You can think of the Feature Map as a form of "materialized memory". If you've been using agents for a while you're probably familiar with the concept of memory - typically these might be stored as simple markdown files (eg an Obsidian vault), or even something more

complex like a vector database. Personally, I think your codebase is the ultimate form of memory. Code is a projection of the decision making you and your team have made and represents the source of truth for what's happened and how things actually work. A Feature Map is just a more compact form of that, designed to save tokens. And because it's just markdown inside of a skill, everyone contributing to your codebase benefits from this shared memory.

你可以把特征图看作是一种“物化的记忆”。如果你使用智能体一段时间了，你可能已经熟悉记忆的概念——通常这些记忆会以简单的 Markdown 文件（例如 Obsidian Vault）的形式存储，或者更复杂的形式，例如矢量数据库。就我个人而言，我认为你的代码库才是记忆的终极形式。代码是你和你的团队决策的体现，代表了事件发生的真实来源以及事物实际运作的方式。特征图只是这种更紧凑的形式，旨在节省令牌。而且因为它只是技能内部的 Markdown 代码，所以所有为你的代码库做出贡献的人都能从这种共享记忆中受益。

This means that maintaining the verification skill is really important. I recommend running `/maintain-verification-skill` at least once a day to ensure that your agents always have the latest details on controlling your app. You may also find, as you use your verification skill more, that agents will automatically update them as they work on your app. `/maintain-verification-skill` catches whatever is missed.

这意味着维护验证技能至关重要。我建议每天至少运行一次 ``/maintain-verification-skill`` 命令，以确保您的客服人员始终掌握最新的应用控制信息。随着您使用验证技能的次数增多，您可能会发现客服人员在使用您的应用时会自动更新验证技能。``/maintain-verification-skill`` 命令可以修复任何遗漏的信息。

How to use your verification skill

如何运用你的验证技能

Here's how I typically use it with pstack.

以下是我通常如何使用 pstack 的方法。

First, of course, is to start your prompt with `/poteto-mode`. If you're using pstack through Cursor, you can also hit `Opt + Enter` instead of just `Enter` when you autocomplete `/poteto-mode` - this adds the skill as a

[Custom Mode](#)

[自定义模式](#)

, which pins the skill so your agent gets a reminder to use the skill on every new turn.

Type `/poteto-mode` and hit `Opt + Enter` to pin it as a Custom Mode

输入 `/poteto-mode` 并按 `Opt + Enter` 将其固定为自定义模式

You can use pstack in Grok Bot too!

你也可以在 Grok Bot 中使用 pstack !

Example: Building new features

示例：构建新功能

For building new features, I typically use the verification skill alongside `/poteto-mode` to get the agent to verify its work. For example, I might prompt something like:

为了构建新功能，我通常会结合使用验证技能和 `/poteto-mode` 命令，让代理验证其工作成果。例如，我可能会提示类似这样的信息：

```
/poteto-mode build <description of feature, any useful context>. use /control-app to
verify your changes and show me a video and screenshots as proof
/poteto-mode 构建<description of feature, any useful context>请使用 /control-app 命
令验证您的更改，并向我提供视频和屏幕截图作为证明。
```

With `/control-app` being the result of `/create-verification-skill`. In Grok

[@Bot](#)

, I would prompt something like:

```
spawn a cloud agent to use /poteto-mode to build <description of feature, any useful
context>. use /control-app to verify your changes and show me a video and
screenshots as proof
生成一个云代理，使用 /poteto-mode 进行构建<description of feature, any useful
context>请使用 /control-app 命令验证您的更改，并向我提供视频和屏幕截图作为证
明。
```

The minor difference here is that in Grok

[@Bot](#)

you tell your bot to spawn a cloud agent instead of doing the work itself. The main reason I prefer to do this is because it frees up your bot to do other things and keeps its context window clean. In that sense, I think of my bots more as coordinators who manage and supervise cloud agents. Cloud agents also mean that you can take advantage of the full array of models available in Cursor which have their own separate machine, so your bot's computer stays free for other things.

Example: Perf work

示例：穿孔工作

```
spawn a cloud agent to use /poteto-mode to improve the initial loading time of our
app. first use /control-app to take a trace of the status quo, and identify opportunities
for improvement. then do a targeted fix and use /control-app + a
/swarm
/群
to confirm the win
```

[/swarm](#)

[/群](#)

is one of the best skills to combine with your verification skill. It fans out any number of cloud agents to run your verification skill, so you can do things like confirm a perf win with a big enough sample size, or fuzz your app to ensure you didn't break or regress anything. Example: Automatically reproduce user reports

示例：自动重现用户报告

When you're happy with your verification skill, you can put them inside of Grok

[@Bot](#)

routines, or

[Cursor Automations](#)

. Routines and automations let you run things on schedule, or trigger whenever an event happens.

For example, if you pipe in user feedback into Slack, and/or have your own internal feedback channel, you can have your bots listen to every report and automatically try to reproduce them with a cloud agent. If your verification skill and Feature Map is good enough, you may even then decide to auto-fix issues as well.

例如，如果您将用户反馈导入 Slack，或者拥有自己的内部反馈渠道，您可以让机器人监听每一条反馈，并自动尝试使用云代理重现问题。如果您的验证技能和功能地图足够完善，您甚至可以选择自动修复问题。

There's a reason I said earlier that verification is the one of the most important skills in your toolbox. It gives you a foundation to build new skills and routines on top of. And most importantly, everyone in your team benefits.

我之前说过，验证是你工具箱里最重要的技能之一，这是有原因的。它能为你构建新的技能和流程打下基础。更重要的是，团队里的每个人都能从中受益。

Invest in your verification skill 投资提升你的验证技能

Once you've created your verification skill, keep it sharp with `/maintain-verification-skill`. Keep improving the CLI and invest in the skill like you would critical infra. You may even want to put an oncall rotation on it - that's how important it is to unlock 100-1000x productivity for your team.

一旦你掌握了验证技能，就使用 ``/maintain-verification-skill`` 命令来保持它的熟练度。不断改进命令行界面 (CLI)，并像对待关键基础设施一样投资于这项技能。你甚至可以考虑安排轮值人员来维护它——它对提升团队 100 到 1000 倍的生产力至关重要。

This skill is the foundation for many other skills that we'll cover in the pstack guide, and composes beautifully with all of them.

这项技能是我们在 pstack 指南中将要介绍的许多其他技能的基础，并且可以与所有这些技能完美地结合使用。

I recommend adding

[Dr Eggbot](#)

[蛋机器人博士](#)

, my bot that helps you create high quality bots, to your roster. Dr Eggbot ships with pstack. It'll teach coding bots how to use it, and it can also make non-coding bots with the same rigor.

You can ask Dr Eggbot to create an engineer bot for you that you can then ask to run `/create-verification-skill` and set up a daily routine to run `/maintain-verification-skill`.

您可以要求 Eggbot 博士为您创建一个工程师机器人，然后您可以要求该机器人运行 `/create-verification-skill` 命令，并设置一个每日例程来运行 `/maintain-verification-skill` 命令。

Thanks for reading and stay tuned for Part 2!

感谢阅读，敬请期待第二部分！